Automation Platform & Ansible Controller Setup

Overview

This document details the comprehensive setup and configuration of the **Ansible automation platform** managing a hybrid infrastructure environment. The automation controller is strategically deployed within the **Management VLAN** and provides centralized configuration management, orchestration, and infrastructure automation capabilities across both Linux and Windows systems throughout the entire lab environment.

Deployment Objectives

Primary Goals

- Establish centralized automation and configuration management platform
- Deploy Ansible controller with comprehensive lab coverage across multiple platforms
- Configure authentication mechanisms for both Linux (SSH) and Windows (WinRM) systems
- Create foundation for infrastructure as code practices
- Enable automated deployment and maintenance workflows across heterogeneous environments

Strategic Implementation

- **VLAN Assignment**: Management VLAN (VLAN 10 - `192.168.10.0/24`)
- **Platform Choice**: VMware Workstation Pro for isolation and flexibility
- **Operating System**: Ubuntu Desktop 24.04 LTS for GUI management capabilities
- **Network Architecture**: Bridged networking for direct VLAN access
- **Multi-Platform Support**: Linux and Windows system management

Virtualization Environment Setup

Host System Configuration

Component	Details
Host OS	Windows 11 Professional
Virtualization	VMware Workstation Pro
Network Adapters	Intel Wi-Fi 6E AX211, Dell Gigabit Ethernet
Target NIC	Dell Gigabit Ethernet (for VLAN bridging)

Component	Details
VM Platform	VMware custom VM configuration

VMware Network Configuration

Virtual Network Editor Setup

The VMware Workstation Pro network configuration was customized to provide direct access to the Management VLAN:

Network Configuration Process: 1. **Opened**: VMware Workstation Pro → **Edit** → **Virtual Network Editor** 2. **Created**: New bridged network named `Mgmt VLAN` 3. **Bridge Target**: Explicitly bridged to **Dell Gigabit Ethernet** adapter 4. **DHCP Configuration**: Disabled VMware DHCP to prevent conflicts 5. **VM Assignment**: Assigned `Mgmt VLAN` (VMnet4) to controller VM

Network Benefits: - **Direct VLAN Access**: VM communicates directly with Management VLAN - **No Host Interference**: Windows DHCP/networking doesn't interfere - **Native Switching**: Traffic handled by physical switch infrastructure - **Security Isolation**: VM properly isolated within VLAN boundaries

Virtual Machine Provisioning

VM Specifications & Configuration

Component	Specification
VM Name	`ansible-controller`
Operating System	Ubuntu Desktop 24.04.2 LTS
CPU Allocation	2 vCPUs
Memory	4GB RAM
Storage	60GB (Thin provisioned)
Network	`Mgmt VLAN` (VMnet4)
Username	`nantwi`

Ubuntu Installation Process

Installation Configuration: - **Hostname**: `ansible-controller` - **Primary User**: `nantwi` - **Installation Type**: Desktop environment with standard utilities - **Security**: Standard Ubuntu security configuration - **Network**: Automatic configuration via VMware bridged networking

Post-Installation Verification: - **System Boot**: Ubuntu Desktop fully operational - **Network Detection**: Interface detected and available - **User Account**: Primary user account functional - **VMware Tools**: Integration tools installed and operational

Network Configuration Implementation

Static IP Configuration Strategy

Target Configuration Requirements

For a production automation controller, predictable network addressing is essential:

Parameter	Value	Justification
IP Address	`192.168.10.2/24`	Reserved static range (`.2` - `.49`)
Gateway	`192.168.10.1`	pfSense Management VLAN gateway
DNS Servers	`8.8.8.8` , `1.1.1.1`	Reliable external DNS for package management
Interface	`ens33`	Primary VMware virtual ethernet

Modern Netplan Configuration Implementation

```
# /etc/netplan/50-cloud-init.yaml - Final configuration
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.10.2/24
      routes:
        - to: 0.0.0.0/0
          via: 192.168.10.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Configuration Application & Verification

```
# Apply network configuration
sudo netplan apply

# Fix file permissions warning
sudo chmod 600 /etc/netplan/*.yaml

# Verify IP assignment and connectivity
ip addr show ens33
ping -c 4 192.168.10.1  # Gateway test
ping -c 4 8.8.8.8       # Internet connectivity test
```

Configuration Results: - **Static IP Assignment**: `192.168.10.2/24` successfully applied - **Gateway Connectivity**: pfSense gateway reachable - **DNS Resolution**: External name resolution functional - **Internet Access**: Package repositories accessible

SSH Service Configuration

SSH Server Installation & Configuration

```
# Install OpenSSH server
sudo apt install -y openssh-server

# Enable SSH service for automatic startup
sudo systemctl enable --now ssh

# Verify SSH service status
sudo systemctl status ssh
```

SSH Configuration Results: - **Service Installation**: OpenSSH server installed and configured - **Automatic Startup**: Service enabled for boot-time activation - **Local Connectivity**: SSH accepting connections on port 22 - **Authentication**: Both key-based and password authentication available

Ansible Installation & Configuration

Ansible Installation Process

```
# Update package repository
sudo apt-get update

# Install required dependencies
sudo apt-get install -y lsb-release software-properties-common gnupg

# Add official Ansible PPA
sudo apt-add-repository -y ppa:ansible/ansible

# Update package list with new repository
sudo apt-get update

# Install Ansible
sudo apt-get install -y ansible
```

Installation Results: - **Latest Version**: Ansible installed from official PPA - **Complete Installation**: All required components and dependencies installed - **Configuration Files**: Default configuration files created - **Command Line Tools**: All Ansible utilities available and functional

Current Infrastructure Overview

Multi-Platform Managed Environment

The Ansible automation platform now successfully manages both Linux and Windows systems, demonstrating enterprise-grade hybrid infrastructure automation capabilities.

Infrastructure Mapping

```
Ansible Controller (192.168.10.2) - Ubuntu 24.04
├── Linux Infrastructure (SSH + ansible service account)
│   ├── 192.168.20.2 (Wazuh SIEM - Rocky Linux 9.6)
│   ├── 192.168.60.2 (Monitoring - Ubuntu 24.04)
│   ├── 192.168.10.4 (TCM Ubuntu - Ubuntu 24.04)
│   └── 192.168.10.2 (Controller - Ubuntu 24.04)
└── Windows Infrastructure (WinRM + ansible user)
    └── 192.168.10.3 (Windows 11 Pro Host)
```

Platform Support Matrix

Platform	Systems	Connection Method	Authentication	Management Scope
Linux	4 systems	SSH	Service account (ansible)	Full system management
Windows	1 system	WinRM	Local user (ansible)	Full system management
Total	**5 systems**	**Mixed protocols**	**Platform-appropriate**	**Unified automation**

Ansible Inventory Configuration

Comprehensive Inventory Architecture

The inventory supports both platform-specific and unified operations:

```
# /etc/ansible/hosts

# Platform-specific groups
[all_in_one]
192.168.20.2   # Wazuh SIEM - Rocky Linux
192.168.60.2   # Monitoring - Ubuntu
192.168.10.4   # TCM Ubuntu
192.168.10.2   # Controller

[windows]
192.168.10.3   # Windows 11 Pro Host

# Function-based groups
[controller]
```

```
192.168.10.2

[wazuh]
192.168.20.2

[grafana]
192.168.60.2

[tcm]
192.168.10.4

# OS-specific groups
[ubuntu]
192.168.60.2   # Grafana/Prometheus
192.168.10.4   # TCM Ubuntu
192.168.10.2   # Controller

[rocky]
192.168.20.2   # Wazuh SIEM

# Unified management group
[all_systems:children]
all_in_one
windows

# Location-based groups
[management_vlan]
192.168.10.2   # Controller
192.168.10.3   # Windows Host
192.168.10.4   # TCM Ubuntu
```

Authentication Configuration

Linux Systems Authentication

Service Account Creation

Professional automation requires dedicated service accounts separate from personal user accounts:

```
# Create ansible service account on each Linux system
sudo useradd -m -s /bin/bash ansible
sudo usermod -aG sudo ansible

# Configure passwordless sudo
echo "ansible ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/ansible
```

SSH Key Management

```
# Generate ED25519 SSH key pair
ssh-keygen -t ed25519 -C "ansible-controller@lab-infrastructure"

# Distribute public key to all Linux systems
ssh-copy-id -i ~/.ssh/id_ed25519.pub ansible@192.168.20.2  # Wazuh
ssh-copy-id -i ~/.ssh/id_ed25519.pub ansible@192.168.60.2  # Monitoring
ssh-copy-id -i ~/.ssh/id_ed25519.pub ansible@192.168.10.4  # TCM Ubuntu
ssh-copy-id -i ~/.ssh/id_ed25519.pub ansible@192.168.10.2  # Controller
```

Windows Systems Authentication

Local User Account Configuration

```
# Create dedicated ansible user on Windows system
net user ansible "SecurePassword123!" /add
net localgroup administrators ansible /add

# Enable WinRM for remote management
winrm quickconfig -force
winrm set winrm/config/service/auth @{Basic="true"}
winrm set winrm/config/service @{AllowUnencrypted="true"}
```

Ansible Configuration for Windows

```
# /etc/ansible/host_vars/192.168.10.3.yml
ansible_connection: winrm
ansible_winrm_transport: basic
ansible_winrm_server_cert_validation: ignore
ansible_user: ansible
ansible_password: SecurePassword123!
```

Cross-Platform Automation Capabilities

Unified Management Commands

```
# Platform-specific connectivity testing
ansible all_in_one -m ping        # Linux systems (4 systems)
ansible windows -m win_ping       # Windows systems (1 system)

# Cross-platform system information
ansible all_in_one -m shell -a "uname -a && uptime"
ansible windows -m win_shell -a "Get-ComputerInfo | Select-Object WindowsProductName,TotalPhysical
```

Service Management Examples

```
# Linux service management
ansible all_in_one -m systemd -a "name=ssh state=restarted"
ansible wazuh -m systemd -a "name=wazuh-manager state=restarted"
ansible grafana -m systemd -a "name=grafana-server state=restarted"

# Windows service management
ansible windows -m win_service -a "name=WinRM state=restarted"
ansible windows -m win_service -a "name=Spooler state=started"
```

Software Management

```
# Linux package management
ansible ubuntu -m apt -a "name=curl state=present"
ansible rocky -m dnf -a "name=curl state=present"

# Windows software management
ansible windows -m win_chocolatey -a "name=git state=present"
```

Ansible Configuration Management

Global Configuration

```
# /etc/ansible/ansible.cfg
[defaults]
inventory = hosts
private_key_file = ~/.ssh/id_ed25519
remote_user = ansible
interpreter_python = auto_silent
host_key_checking = False
timeout = 30

[ssh_connection]
ssh_args = -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no
```

Group Variables Configuration

```
# /etc/ansible/group_vars/all_in_one.yml
ansible_user: ansible
ansible_ssh_private_key_file: ~/.ssh/id_ed25519
ansible_become: true
ansible_become_method: sudo

# /etc/ansible/group_vars/windows.yml
ansible_connection: winrm
ansible_winrm_transport: basic
```

```
ansible_winrm_server_cert_validation: ignore
ansible_port: 5985
```

Wazuh Agent Management Integration

Wazuh Repository Management

To manage Wazuh agent automatic updates, the repository configuration can be controlled via Ansible:

```
# Disable automatic updates on managed nodes
ansible wazuh -m shell -a "sed -i 's/^enabled=1/enabled=0/' /etc/yum.repos.d/wazuh.repo"

# Verify repository status
ansible wazuh -m shell -a "grep enabled /etc/yum.repos.d/wazuh.repo"
```

This allows centralized control of when Wazuh components are updated across the infrastructure.

Sample Playbooks

Cross-Platform Ping Playbook

```
# playbooks/ping_all_systems.yml
---
- name: Ping all Linux systems
  hosts: all_in_one
  tasks:
    - name: Ping Linux systems
      ping:

- name: Ping all Windows systems
  hosts: windows
  tasks:
    - name: Ping Windows systems
      win_ping:
```

System Information Gathering

```
# playbooks/system_info.yml
---
- name: Gather Linux system information
  hosts: all_in_one
  tasks:
    - name: Get system information
      shell: |
        echo "Hostname: $(hostname)"
        echo "OS: $(lsb_release -d | cut -f2)"
```

```
        echo "Kernel: $(uname -r)"
        echo "Uptime: $(uptime -p)"
      register: linux_info

    - name: Display Linux information
      debug:
        var: linux_info.stdout_lines

- name: Gather Windows system information
  hosts: windows
  tasks:
    - name: Get Windows information
      win_shell: |
        $info = Get-ComputerInfo
        "Hostname: $($info.CsName)"
        "OS: $($info.WindowsProductName)"
        "Version: $($info.WindowsVersion)"
        "Uptime: $((Get-Date) - $info.CsBootUpTime)"
      register: windows_info

    - name: Display Windows information
      debug:
        var: windows_info.stdout_lines
```

Current Operational Status

Connectivity Validation Results

```
# Cross-platform connectivity verification
$ ansible-playbook playbooks/ping_all_systems.yml

PLAY RECAP *********************************************************
192.168.10.2               : ok=7    changed=0    unreachable=0    failed=0
192.168.10.3               : ok=7    changed=0    unreachable=0    failed=0   ✅ Windows
192.168.10.4               : ok=7    changed=0    unreachable=0    failed=0
192.168.20.2               : ok=7    changed=0    unreachable=0    failed=0
192.168.60.2               : ok=7    changed=0    unreachable=0    failed=0
```

Platform-Specific Operations

```
# Linux system management validation
$ ansible all_in_one -m shell -a "sudo whoami"
192.168.20.2 | CHANGED | rc=0 >> root
192.168.60.2 | CHANGED | rc=0 >> root
192.168.10.4 | CHANGED | rc=0 >> root
192.168.10.2 | CHANGED | rc=0 >> root

# Windows system management validation
```

```
$ ansible windows -m win_shell -a "whoami"
192.168.10.3 | CHANGED | rc=0 >> desktop-abc123\ansible
```

Security Implementation

Authentication Security Standards

Account Type	Linux Systems	Windows Systems	Purpose
Personal Account	`nantwi` (SSH)	Personal Windows user	Development, administration
Automation Account	`ansible` (SSH, passwordless sudo)	`ansible` (WinRM, Administrator)	Automated operations

Network Security

- **VLAN Isolation**: Controller properly isolated in Management VLAN
- **Firewall Controls**: pfSense manages all inter-VLAN communication
- **SSH Security**: Key-based authentication for Linux systems
- **WinRM Security**: Dedicated local administrator for Windows systems
- **Access Control**: Administrative functions restricted to Management VLAN

Current Automation Capabilities

Immediate Operational Capabilities

- **Cross-Platform Management**: 5 systems across Linux and Windows platforms
- **Unified Inventory**: Single inventory managing heterogeneous environment
- **Protocol Compatibility**: SSH (Linux) and WinRM (Windows) simultaneously
- **Service Management**: Platform-appropriate service control across all systems

Ready for Implementation

- **Configuration Management**: Standardize configurations across platforms
- **Software Deployment**: Automated application installation across Linux and Windows
- **Security Hardening**: Apply security configurations consistently
- **Monitoring Setup**: Deploy monitoring agents across all platforms
- **Backup Automation**: Implement automated backup procedures

Operational Validation

Technical Validation

- **Network Configuration**: Static IP properly configured and stable

- **Cross-Platform Connectivity**: Passwordless access to all managed systems
- **Ansible Functionality**: All Ansible tools operational across platforms
- **Inventory Management**: Systems properly organized and accessible
- **Authentication Methods**: Platform-appropriate authentication working

Security Validation

- **SSH Key Security**: Modern cryptography and proper key management
- **WinRM Security**: Dedicated service accounts for Windows systems
- **Network Isolation**: Proper VLAN isolation maintained
- **Access Controls**: Administrative access appropriately restricted
- **Audit Trail**: Clear separation between personal and automated operations

Future Enhancement Pipeline

Planned Automation Workflows

- **Unified Security Hardening**: Cross-platform security configuration enforcement
- **Hybrid Monitoring**: Unified monitoring agent deployment across platforms
- **Configuration Management**: Standardized configurations for mixed environments
- **Compliance Automation**: Automated compliance checking across platforms

Enterprise Integration Opportunities

- **Active Directory Integration**: Windows domain authentication with Linux systems
- **Certificate Management**: Unified PKI deployment across platforms
- **Backup Strategies**: Cross-platform backup and recovery automation
- **Disaster Recovery**: Platform-agnostic recovery procedures

Cross-Platform Best Practices Established

Automation Architecture

- **Conditional Logic**: OS-specific task execution in playbooks
- **Group Variables**: Platform-specific connection configuration
- **Module Selection**: Appropriate modules for each platform
- **Unified Inventory**: Single inventory managing mixed environments

Security Standards

- **Platform-Appropriate Authentication**: SSH keys for Linux, WinRM for Windows
- **Service Account Separation**: Dedicated automation accounts on both platforms
- **Network Security**: Consistent VLAN security controls across platforms
- **Audit Capabilities**: Clear separation of automated vs manual operations

Implementation Summary

Successfully Implemented Components

Component	Status	Configuration	Access Method
VMware Network	Operational	Bridged to Management VLAN	Direct VLAN access
Ubuntu VM	Operational	4GB RAM, 2 vCPU, 60GB disk	Console + SSH
Static Network	Operational	`192.168.10.2/24`	Netplan configuration
SSH Service	Operational	Port 22, key-based auth	`ssh nantwi@192.168.10.2`
Ansible Platform	Operational	Latest version, PPA install	Command line tools
Linux Authentication	Operational	SSH keys + service accounts	4 Linux systems
Windows Authentication	Operational	WinRM + local administrator	1 Windows system
Cross-Platform Inventory	Operational	5 systems, multiple groups	Mixed protocols

Infrastructure Management Achievement

- **Multi-Platform Control**: Automation controller managing Linux and Windows systems
- **Professional Authentication**: Platform-appropriate service accounts
- **Network Segmentation**: Proper VLAN isolation maintained across platforms
- **Centralized Control**: Single point of automation control for heterogeneous environment
- **Enterprise Standards**: Professional-grade automation practices demonstrated

---

Automation Platform Status: Complete and Operational
Infrastructure Management: 5 Systems Across 2 Platforms
Next Phase: Advanced Cross-Platform Automation Workflows